DEVITAPE CONNECT | 12/09/2026

Calma, não surta!

Automação open source para engenheiros sobreviverem ao FinOps

Felipe KiKo

A fatura chegou...

E lá vai você exportar CSV, limpar planilha, caçar TAG errada...de novo, de novo, e de novo...

Quem é o KiKo?

- **Solution Architect** @ Stacklet
- **AWS Community Builder**
- Leader do **AWS User Group Campinas**
- **FinOps and AWS Certified**

Movido a curiosidade, Pokémon, café e...
alguns sustos!



1. O Problema

O trabalho manual de FinOps que nunca acaba

O dia a dia de quem faz FinOps na mão

A ROTINA

- Exportar fatura da nuvem
- Baixar CUR, abrir no Excel
- Limpar dados, cruzar planilha
- Caçar quem esqueceu recurso ligado
- Mandar mensagem *"hey, é seu isso?"*

O RESULTADO

- Sexta-feira inteira nisso
- Relatório já **desatualizado** ao terminar
- Ninguém corrige as tags
- Custo continua subindo
- E o café? **Esfriou.**

Trabalho repetitivo não é FinOps. É **sofrimento manual com nome bonito.**

Sem automação = ciclo sem fim

1. **Não é one-shot**

   Você arruma hoje. Amanhã já tem recurso novo, TAG nova errada, gasto novo.

2. **Tudo muda o tempo todo**

   Cloud é viva. Time cria, deleta, escala. O retrato de ontem já venceu.

3. **Os "esquecidinhos"**

   Volume órfão, IP parado, snapshot de 2023, ambiente de teste zumbi...ninguém lembra.

> *É como capinar mato: parou de capinar, o mato volta!*

2. A Virada de Jogo

Se você faz na mão mais de uma vez, a máquina deveria estar fazendo por você...

Trabalho manual cobra caro

⏳ CUSTA TEMPO

Horas de engenheiro caro fazendo planilha em vez de resolver problema de verdade.

🐛 CUSTA ERRO

Copia-cola, TAG errada, recurso deletado por engano. Humano cansado erra.

🐌 CUSTA ATRASO

Quando o relatório fica pronto, o gasto já aconteceu. Você só olha o retrovisor.

😵 CUSTA ENERGIA

Ninguém quer ser o "xerife das TAGs". Tarefa chata desmotiva o time todo.

FinOps, na real, são 3 movimentos

👀 VISIBILIDADE

Saber o que existe, quanto custa e de quem é. **Sem isso, nada mais funciona.**

✂️ OTIMIZAÇÃO

Achar desperdício, ajustar tamanho, desligar o que não usa.

🔁 OPERAÇÃO

Manter rodando sozinho, no dia a dia, sem depender de herói.

A automação entra nos **três**, mas começa pela **visibilidade**

3. O Kit Open Source

Ferramentas grátis, sem lock-in, rodando do seu jeito, onde você quiser!

O kit open source 🧰

- Cloud Custodian
- CloudQuery
- Steampipe
- Infracost
- Redash
- Python

Cloud Custodian: Governança como código

O QUE FAZ?

Você descreve a regra em YAML, ele **detecta, avisa e corrige**...sozinho, no agendamento

POR QUE IMPORTA?

Para o problema na origem. Sem TAG? Avisa. Ficou órfão? Marca pra limpar!

```yaml
policies:
  - name: nome-da-policy
    resource: aws.ec2
    mode:
      type: periodic
      schedule: rate(1 day)
      ...
    filters:
      - type: value
        key: AlgumaPropriedade
        op: greater-than
        value: 30
    actions:
      - type: notify
        template: alerta
        ...
```

Faxina #1: EBS órfãos

O CENÁRIO

- Instância foi deletada
- O volume EBS... **ficou lá**
- Ninguém lembra que existe
- Cobrando GB/mês em silêncio

A POLICY

$0

de esforço humano
pra achar e limpar

Volume órfão é a caixa de papelão vazia que você paga aluguel de depósito pra guardar...Cloud Custodian joga fora por você!

Faxina #2: EC2 de Dev fora do expediente

```yaml
policies:
  - name: parar-dev-fora-do-expediente
    resource: ec2
    filters:
      - "tag:Environment": dev
      - State.Name: running
    actions:
      - stop
```

- Dev usado ~8h/dia
- Ligado **24h/dia**
- Você paga por **16h ociosas**

~65%

só desligando fora do horário

Faxina #3: RDS grande demais pra Dev

O QUE A POLICY ACHA

- RDS `db.r5.xlarge` em **dev**
- CPU média: **4%**
- "Copiei da produção..."
- Multi-AZ ligado (2x custo!)

O QUE ELA FAZ

- **Notifica** o time com a evidência
- Sugere `db.t3.medium` + Single-AZ
- Marca com TAG `revisar-custo`
- Vira card no board automaticamente

Ambiente de Dev não precisa aguentar a Black Friday, e a policy lembra o time disso **sem apontar dedo**

Faxina #4: A caça às TAGs perdidas

```yaml
policies:
  - name: recursos-sem-dono
    resource: ec2
    filters:
      - "tag:Owner": absent      # cadê o dono?
    actions:
      - type: tag
        key: finops-status
        value: sem-dono-revisar
      - type: notify
        to: ["#finops-alerts"]
```

Sem TAG = sem dono. Sem dono = ninguém corta. Sem corte = fatura sobe.

TAG é o GPS do seu dinheiro na nuvem.

CloudQuery: Visibilidade em query

O QUE FAZ

Extrai o inventário da nuvem (AWS, Azure, GCP...) e joga tudo num banco SQL

POR QUE IMPORTA

Se está em SQL, você **pergunta**. "Quais volumes não têm dono?" vira um `SELECT`

```sql
-- Volumes EBS órfãos (sem instância anexada)
SELECT id, size, availability_zone
FROM aws_ec2_ebs_volumes
WHERE state = 'available';
```

Steampipe: SQL na sua nuvem

```sql
-- "Quais EBS não estão anexados a nada?"
SELECT volume_id, size, region
FROM aws_ebs_volume
WHERE attachments IS NULL;
```

POR QUE É MÁGICO

- Consulta a nuvem como se fosse **banco de dados**
- Sem exportar CSV
- Sem abrir console
- Resposta em **segundos**

O QUE DÁ PRA PERGUNTAR

- Recursos sem TAG
- Volumes órfãos
- Buckets públicos
- IPs elásticos parados

CloudQuery e Steampipe: inventário completo e versionado!

O QUE FAZ

- Extrai **todo** o inventário da nuvem
- Joga num banco (Postgres, etc.)
- Você cruza dados com **SQL** ou BI
- Roda todo dia, guarda histórico

O QUE VOCÊ GANHA

- Dashboard de custo por time
- Histórico: "cresceu quando?"
- Base pra DataViz
- Fonte da verdade única

Visibilidade não é dashboard bonito. É conseguir **responder perguntas** rápido.

Infracost: Custo no Pull Request

```
# No PR, antes do merge, o bot comenta:

+ aws_instance.app (t3.xlarge)      +$121/mês
+ aws_nat_gateway.main              +$ 32/mês
+ aws_db_instance.db (Multi-AZ)     +$280/mês
──────────────────────────────────────────────
  Este PR aumenta o custo em        ~$433/mês
```

É o **shift-left de FinOps**: a pergunta "quanto isso vai custar?", e aparece no momento do CI/CD, não na fatura 30 dias depois

Redash: SQL vira dashboard

O QUE FAZ?

Conecta no banco do **CloudQuery / Steampipe**, você escreve **SQL** e ele vira gráfico, tabela e dashboard compartilhável.

POR QUE IMPORTA?

Transforma dado bruto em **resposta**. Cada time abre o painel e **BOOM!** se serve de informações sem chamar você

```sql
-- Top 10 recursos mais caros por time
SELECT
  tags ->> 'time'   AS time,
  service,
  SUM(custo_mes)    AS total
FROM fatura_aws
GROUP BY 1, 2
ORDER BY total DESC
LIMIT 10;
```

Salve como *query*, plote o gráfico e fixe no dashboard, agende o refresh e pronto! **BI** facin e rapidin!

Quando nada se encaixa...Python (ou X)

Nenhuma das anteriores resolveu **exatamente** o seu caso? Tudo bem. Um bom e velho script simples faz o mesmo trabalho!

```python
# 30 linhas de Python já resolvem:
import boto3

ce = boto3.client("ce")
custo = ce.get_cost_and_usage(...)      # puxa a fatura

for time, valor in agrupa_por_tag(custo):
    if valor > LIMITE:
        notifica_slack(time, valor)     # avisa o dono
```

A ferramenta é meio, não fim. Se um cron + 30 linhas de Python resolvem, pronto!

4. O Checklist

O que automatizar PRIMEIRO (sem tentar conquistar o mundo)

Comece por aqui 👇

1. **Inventário automático**
   O que existe e de quem é: CloudQuery / Steampipe
2. **Alerta de TAG faltando**
   Pare o problema na origem, antes de virar bagunça.
3. **Achar o desperdício óbvio**
   Disco, IP e snapshot órfão, o dinheiro fácil de resgatar.
4. **Notificação pro dono**
   No canal que ele já usa (Slack, e-mail, Teams).
5. **Remediação com trava**
   Parar / limpar só DEPOIS de avisar. Nunca no susto.

5. E o Kiro nisso tudo?

IA acelerando cada passo do checklist

Onde o Kiro entra

ESCREVE A AUTOMAÇÃO COM VOCÊ

Gera a policy do Cloud Custodian, o SQL do CloudQuery e o script Python de cola...a partir do que você quiser!

EXPLICA ANTES DE APLICAR

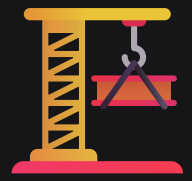
Mostra custos, o que a regra faz, o que vai afetar e sugere a trava de segurança antes de sair remediando

Não é substituir você! É ganhar um **companheiro nerd** que nunca reclama do café!

Kiro na prática

```
Você:  "Cria uma policy do Custodian que marca EBS órfão
        e avisa o dono antes de deletar em 7 dias."

Kiro:  "Feito! Montei com filtro de volume 'available',
        notify pro Owner e delete com trava de 7 dias.
        Quer que eu adicione um dry-run primeiro?"
```



Gera policy, SQL
e Scripts



Analisa e sugere travas
e dry-run (importante!)



Ajuda a acelerar o
checklist do 1 ao 5

Do papel ao piloto automático

SEM KIRO

Ler documentações, lembrar a sintaxe, testar na mão e torcer pra não deletar nada errado...

COM KIRO

Descreve a intenção, revisa o que ele gerou, aplica com trava. Foco no **o quê**, não no **como**.

Call to Action

Uma automação...só uma...e amanhã!

Escolha UMA coisa pra automatizar

COMECE

Rode o CloudQuery e faça **um** SELECT de órfãos.

AUTOMATIZE

Escreva **uma** policy de TAG faltando.

MOSTRE

Suba **um** dashboard pro time enxergar o custo.

Você não precisa virar FinOps engineer. Precisa **parar de fazer na mão** o que a máquina faz melhor.

A regra de ouro

> *Comece pequeno.* ***Uma política. Um relatório. Uma vitória.***

1

Política que funciona
> 10 no papel

1

Relatório que roda
sozinho toda semana

1

Vitória que o time
vê e confia

Sábias palavras...

🔁 **Manual não escala, automação sim.**

Cloud muda todo dia. Sua resposta tem que rodar sozinha.

👀 **Comece pela visibilidade.**

Se está em SQL, você pergunta. Se você pergunta, você decide.

🧰 **Open source resolve, e é de graça.**

CloudQuery + Cloud Custodian + Python + Dashboards. Sem lock-in.

👻 **Use o Kiro pra acelerar.**

Ele escreve a automação com você e sugere a trava de segurança.

🎯 **Comece pequeno, uma vitória por vez.**

Uma política. Um relatório. Depois +1, e +1...

Referências e links úteis

- Cloud Custodian: `cloudcustodian.io`
- CloudQuery: `cloudquery.io`
- Steampipe: `steampipe.io`
- Infracost: `infracost.io`
- Redash: `redash.io`
- Kiro: `kiro.dev`
- FinOps Foundation: `finops.org`

> *FinOps não é sobre **cortar o cafezinho**, é sobre **automatizar a chatice** pra sobrar tempo de tomar ele tranquilo!* ☕



Dúvidas? Obrigado!